

Komplexaufgabe zur Algorithmmierung

Informatik · Klasse 9 · Arbeitsheft

Inhaltsverzeichnis

01_Arbeitsheft	5
Was ist ein Algorithmus?	6
Einstieg	6
Praxis	6
Theorie	6
Vergleiche eure Ergebnisse	6
Entdecken	7
Eine Anleitung wird ausprobiert	7
Praxis	7
Theorie	7
Gespräch	7
KAI fragt	7
Auftrag	8
Erarbeiten	9
Was macht eine gute Anleitung aus?	9
Aufgabe 1	9
Aufgabe 2	9
Bedeutungen	9
KAI erklärt	10
Aufgabe 3	10
Merke	11
Vom Alltag zur Informatik	11
Merke Merksatz	11
KAI erklärt	11
Merksatz anwenden	11
Merksatz sichern	12
Beispiele	13
Beispiel 1 – Größere Zahl bestimmen	13
Beispiel 2 – Summe von 1 bis n	13
Beispiel 3 – Eingabe prüfen	13
Beispiel 4 – Algorithmus testen	13
Transfer	14
Alltag und Computer	15
Warum brauchen Computer Algorithmen?	15
KAI erklärt	15
Mensch oder Computer?	15
Wie versteht ein Computer einen Algorithmus?	15
Scratch und Python	15
Ausblick	16
Algorithmen mit Scratch umsetzen	17
Das Problem	17
Ein möglicher Algorithmus	17
Scratch kennenlernen	17
Schritt für Schritt	17
Programm testen	18
KAI erklärt	18
Ausprobieren	18
Merke	18

Algorithmen mit Python umsetzen	19
Dasselbe Problem	19
Python kennenlernen	19
Das erste Python-Programm	19
Was passiert?	19
Programm testen	19
Scratch und Python im Vergleich	19
KAI erklärt	20
Ausprobieren	20
Merke	20
Übungen - Lineare Algorithmen	21
Ziel	21
1. Erkennen	21
2. Fehler finden	21
3. Reihenfolge	22
4. Vollständigkeit	22
5. Eindeutigkeit	23
6. Einen Algorithmus entwickeln	23
7. Testen	24
8. Verschiedene Lösungen vergleichen	24
9. Scratch und Python	25
10. Fehler erklären	25
11. Eigener Fehleralgorithmus	26
12. Transfer	26
Plus - Effizienz	27
Transfer	28
Aufgabe 1 - Frühstück vorbereiten	28
Aufgabe 2 - Papierflieger falten	28
Aufgabe 3 - Der Schulweg	28
KAI-Challenge	28
Aufgabe 4 - Ist das ein Algorithmus?	29
Aufgabe 5 - Eigene Idee	29
Projekt - KAI übernimmt eine neue Aufgabe	30
Der Auftrag	30
Aufgabe	30
Planung	30
Entwickelt euren Algorithmus	30
Test	31
Überarbeitet euren Algorithmus	31
Präsentation	31
KAI erklärt	31
Reflexion	32
Kompetenzcheck	33
Das kann ich	33
Aufgabe 1 - Algorithmus erkennen	33
Aufgabe 2 - Fehler finden	33
Aufgabe 3 - Algorithmus entwickeln	33
Aufgabe 4 - Darstellungen vergleichen	34
Selbsteinschätzung	34
Geschafft?	34

Projektstart - Von der Übung zur Komplexaufgabe	35
Jetzt beginnt das eigentliche Projekt	35
Was ist ein Projekt?	35
Der Projektauftrag	35
Unser Team	35
Unsere erste Idee	35
Rollen im Team	36
Der Qualitätskreislauf	36
Projektregeln	36
Startcheck	37
Problemanalyse - Was müssen wir eigentlich lösen?	38
Nicht sofort programmieren	38
1. Problem beschreiben	38
2. Ziel beschreiben	38
3. Zielgruppe	38
4. Eingaben, Verarbeitung, Ausgaben und Speicherung	38
5. Anforderungen	39
6. Grenzen festlegen	39
7. Prüfen	40
Lösungsentwurf und Planung	41
Vom Problem zur Lösung	41
1. Lösungsideen sammeln	41
2. Ideen vergleichen	41
3. Algorithmus entwerfen	42
4. Umsetzung wählen	42
5. Projekt in kleine Aufgaben zerlegen	42
6. Erste lauffähige Version	42
Planungscheck	43
Umsetzung - Aus dem Entwurf wird eine funktionierende Lösung	44
Kleine Schritte	44
Version 1	44
Entwicklungsprotokoll	44
Wenn etwas nicht funktioniert	45
KAI fragt	45
Kommunikation im Team	45
Zwischenstand sichern	45
Testen und Verbessern	46
„Es läuft“ reicht nicht	46
1. Testfälle entwickeln	46
Unsere Testfälle	46
2. Muss-Anforderungen testen	46
3. Fremdtest	47
4. Fehler priorisieren	47
5. Qualitätskreislauf	47
Dokumentation - Den Lösungsweg nachvollziehbar machen	49
Dokumentation ist Teil des Projekts	49
Eure Projektdokumentation	49
Entscheidungen dokumentieren	50
Schwierigkeiten	50
Zusammenarbeit	50
Dokumentationscheck	50

Präsentation und Reflexion	51
Euer Ergebnis vorstellen	51
Präsentationsstruktur	51
Präsentationsregeln	51
Vorbereitung	51
Reflexion	53
Über das Ergebnis	53
Über den Lösungsweg	53
Über die Zusammenarbeit	53
Über das Lernen	53
Abschluss	54
Arbeitsheft - Komplexaufgabe 1: Lineare Algorithmen	55
Ausgangssituation	55
Kompetenzziele	55
Aufgabe 1	55
Aufgabe 2	55
Aufgabe 3	55
Aufgabe 4	55
Transfer	55
Kompetenzcheck	55

01_Arbeitsheft

Enthält das Arbeitsheft.

Was ist ein Algorithmus?

Einstieg

In wenigen Wochen findet an eurer Schule das Schulfest statt.

Viele Klassen bereiten verschiedene Angebote vor. In der Schulküche soll Kakao verkauft werden. Damit alle Helferinnen und Helfer den Kakao gleich zubereiten, soll eine Anleitung erstellt werden.

Herr Lehmann hat eine Idee.

„Schreibt eine Anleitung, mit der jede Person einen Becher Kakao zubereiten kann.“

Praxis

Arbeitet zu zweit.

Schreibt eine Anleitung für einen Becher Kakao auf.

Verwendet nur kurze Handlungsschritte.

Theorie

Lies die folgende Aufgabe.

Schreibe eine Anleitung, mit der jede Person einen Becher Kakao zubereiten kann.

Notiere deine Handlungsschritte.

Vergleicht eure Ergebnisse

Tauscht eure Anleitungen mit einer anderen Gruppe.

Lest die Anleitung aufmerksam durch.

Sprecht anschließend darüber.

Fragen zur Diskussion

- Sind alle Anleitungen gleich?
- Gibt es Unterschiede?
- Welche Anleitung lässt sich besonders gut nachvollziehen?
- Gab es Stellen, die unklar waren?

Gespräch **Merke dir deine Beobachtungen.**

Im nächsten Abschnitt untersucht ihr, wodurch eine gute Anleitung entsteht.

Entdecken

Eine Anleitung wird ausprobiert

Eine Gruppe hat folgende Anleitung geschrieben.

Kakao zubereiten

1. Rühre gut um.
 2. Gib zwei Teelöffel Kakaopulver dazu.
 3. Fülle Milch in einen Becher.
-

Praxis

Eine Person liest die Anleitung laut vor.

Eine zweite Person führt die Schritte **genau in der angegebenen Reihenfolge** aus.

Die übrigen Gruppenmitglieder beobachten.

Wichtig:

Es darf **nichts ergänzt oder verbessert** werden.

Es wird ausschließlich das gemacht, was in der Anleitung steht.

Theorie

Lies die Anleitung aufmerksam durch.

Stelle dir vor, du musst sie Schritt für Schritt ausführen.

Notiere alle Stellen, an denen Schwierigkeiten auftreten.

Gespräch

Vergleicht eure Beobachtungen.

Besprecht anschließend folgende Fragen.

- Was ist beim Ausführen passiert?
 - Warum war die Anleitung schwierig umzusetzen?
 - Welche Informationen haben gefehlt?
 - Welche Schritte hätten an einer anderen Stelle stehen müssen?
-

KAI fragt

Ich möchte Schritt 1 ausführen.

Was soll ich umrühren?

Auftrag

Verbessert die Anleitung.

Ändert sie so, dass jede Person den Kakao ohne Rückfragen zubereiten kann.

Vergleicht anschließend eure neue Version mit einer anderen Gruppe.

Gespräch Welche Änderungen habt ihr vorgenommen?

Erarbeiten

Was macht eine gute Anleitung aus?

Ihr habt verschiedene Anleitungen verglichen und verbessert.

Nun stellt sich die Frage:

Wodurch wird eine Anleitung eigentlich gut?

Sammelt eure Ideen in der Klasse.

Aufgabe 1

Ergänzt die Tabelle.

Beobachtung	Was sollte eine gute Anleitung können?
Manche Angaben waren ungenau. Einige Schritte fehlten. Die Reihenfolge war nicht sinnvoll. Manche Anweisungen konnten nicht ausgeführt werden.	

Vergleicht anschließend eure Ergebnisse.

Aufgabe 2

Herr Lehmann fasst die Ergebnisse der Klasse zusammen.

Eine gute Anleitung ist ...

- eindeutig.
- vollständig.
- in der richtigen Reihenfolge.
- ausführbar.

Übertragt die Begriffe in eure Tabelle.

Bedeutungen

Eindeutig

Alle verstehen die Anweisung gleich.

Beispiel

Nein Gib etwas Milch dazu.

Ja Gib 250 ml Milch dazu.

Vollständig

Kein notwendiger Schritt fehlt.

Beispiel

Nein Rühre um.

(Es wurde vorher nichts eingefüllt.)

Richtige Reihenfolge

Die Handlungsschritte bauen logisch aufeinander auf.

Beispiel

Nein Schuhe anziehen.

Nein Socken anziehen.

Ausführbar

Jeder Schritt kann tatsächlich ausgeführt werden.

Beispiel

Nein Fliege zum Mond.

KAI erklärt

Ich kann nur das ausführen, was eindeutig beschrieben wurde.

Aufgabe 3

Ordnet die folgenden Aussagen einer Eigenschaft zu.

Aussage	Eigenschaft
„Gib etwas Zucker dazu.“	
„Backe den Kuchen.“ (Es fehlt das Rezept.)	
„Setze den Deckel auf.“ bevor das Glas gefüllt wurde.	
„Hole den Regenbogen.“	

Vergleicht eure Lösungen.

Merke

Vom Alltag zur Informatik

Ihr habt verschiedene Anleitungen untersucht und verbessert.

Dabei habt ihr festgestellt:

- Anweisungen müssen eindeutig sein.
- Es dürfen keine wichtigen Schritte fehlen.
- Die Reihenfolge muss stimmen.
- Alle Schritte müssen ausführbar sein.

In der Informatik gibt es für eine solche Handlungsvorschrift einen eigenen Begriff.

Merke Merksatz

Ein Algorithmus ist eine eindeutige, vollständige und ausführbare Folge von Handlungsschritten zur Lösung eines Problems.

Die Handlungsschritte müssen in einer sinnvollen Reihenfolge ausgeführt werden.

KAI erklärt

Ich kann einen Algorithmus nur ausführen, wenn alle Handlungsschritte eindeutig beschrieben sind.

Merksatz anwenden

Prüfe die folgenden Aussagen.

Kreuze an und begründe deine Entscheidung.

1.

„Gib etwas Milch in den Becher.“

☐ erfüllt den Merksatz

☐ erfüllt den Merksatz nicht

Begründung:

2.

„Schalte den Computer ein.“

☐ erfüllt den Merksatz

☐ erfüllt den Merksatz nicht

Begründung:

3.

„Öffne die Datei.“

☐ erfüllt den Merksatz

☐ erfüllt den Merksatz nicht

Begründung:

Merksatz sichern

Ergänze den Merksatz.

Ein Algorithmus ist eine

Beispiele

Algorithmen begegnen dir im Alltag und in Computerprogrammen. Entscheidend ist nicht das Thema, sondern dass die Lösung als eindeutige, ausführbare und endliche Folge von Schritten beschrieben werden kann.

Beispiel 1 - Größere Zahl bestimmen

Eingabe: zwei Zahlen a und b.

Pseudocode:

```
wenn a > b dann
    gib a aus
sonst
    gib b aus
ende wenn
```

Teste den Ablauf mit (8, 3), (2, 9) und (5, 5). Was fällt beim letzten Test auf? Ergänze den Algorithmus so, dass Gleichheit ausdrücklich behandelt wird.

Beispiel 2 - Summe von 1 bis n

Für eine positive ganze Zahl n soll die Summe $1 + 2 + \dots + n$ berechnet werden.

```
summe ← 0
für zahl von 1 bis n
    summe ← summe + zahl
ende für
gib summe aus
```

Führe den Algorithmus für $n = 4$ schrittweise in einer Tabelle aus. Notiere nach jedem Schleifendurchlauf zahl und summe.

Beispiel 3 - Eingabe prüfen

Ein Programm darf nur Werte von 1 bis 10 akzeptieren.

```
wiederhole
    lies wert ein
solange wert < 1 oder wert > 10
gib wert aus
```

Erkläre, warum hier eine bedingte Wiederholung sinnvoller ist als eine feste Wiederholungszahl.

Beispiel 4 - Algorithmus testen

Entwirf für Beispiel 1 mindestens drei Testfälle. Ein guter Testsatz enthält neben typischen Werten auch Rand- oder Sonderfälle.

Nutze die Tabelle:

Eingabe	erwartetes Ergebnis	tatsächliches Ergebnis	Änderung nötig?
---------	---------------------	------------------------	-----------------

Transfer

Beschreibe einen eigenen kleinen Algorithmus zunächst in Alltagssprache und anschließend als Pseudocode oder Struktogramm. Markiere Sequenz, Auswahl und Wiederholung, soweit sie vorkommen.

Alltag und Computer

Warum brauchen Computer Algorithmen?

Im Alltag führen Menschen viele Aufgaben aus, ohne über jeden einzelnen Schritt nachzudenken.

Zum Beispiel:

- Zähne putzen
- Einen Schulranzen packen
- Einen Kakao zubereiten

Dabei ergänzen Menschen oft fehlende Informationen automatisch.

Computer können das nicht.

KAI erklärt

Wenn mir Informationen fehlen, kann ich sie nicht erraten.

Ich führe nur das aus, was mir genau gesagt wird.

Mensch oder Computer?

Ordne die Aussagen zu.

Aussage	Mensch	Computer
Kann fehlende Informationen ergänzen.	☐	☐
Führt Anweisungen genau aus.	☐	☐
Erkennt oft, was gemeint ist.	☐	☐
Benötigt eindeutige Anweisungen.	☐	☐

Wie versteht ein Computer einen Algorithmus?

Ein Computer kann einen Algorithmus nicht direkt aus der Alltagssprache ausführen.

Deshalb schreiben Informatikerinnen und Informatiker Algorithmen in einer **Programmiersprache** auf.

Es gibt viele Programmiersprachen.

Zwei Beispiele sind **Scratch** und **Python**.

Scratch und Python

Scratch	Python
Programme werden aus Blöcken zusammengesetzt.	Programme werden als Text geschrieben.
Gut geeignet für den Einstieg.	Weit verbreitete Programmiersprache.

Beide Programmiersprachen können denselben Algorithmus beschreiben.

Ausblick

Im nächsten Abschnitt lernst du zwei Möglichkeiten kennen, einen Algorithmus in einer Programmiersprache umzusetzen.

Zuerst arbeitest du mit **Scratch**.

Anschließend setzt du denselben Algorithmus in **Python** um.

So erkennst du, dass sich nur die Schreibweise ändert – die Lösungsidee bleibt dieselbe.

Algorithmen mit Scratch umsetzen

Das Problem

Beim Schulfest sollen alle Besucher freundlich begrüßt werden.

KAI soll diese Aufgabe übernehmen.

Überlegt zunächst:

Was soll KAI tun?

Notiert einen passenden Algorithmus.

Ein möglicher Algorithmus

1. Warte, bis das Programm gestartet wird.
2. Sage: „Willkommen auf unserem Schulfest!“

Diesen Algorithmus setzen wir nun in Scratch um.

Scratch kennenlernen

Scratch ist eine Programmiersprache.

Programme werden nicht als Text geschrieben, sondern aus Programmblöcken zusammengesetzt.

Jeder Block steht für eine Anweisung.

Mehrere Blöcke ergeben zusammen ein Programm.

Schritt für Schritt

Schritt 1

Starte Scratch.

Erstelle ein neues Projekt.

Schritt 2

Suche den Block

Wenn die grüne Flagge angeklickt wird

Ziehe ihn in den Programmierbereich.

Schritt 3

Füge darunter den Block

Sage „Hallo!“

ein.

Ändere den Text in

Willkommen auf unserem Schulfest!

Programm testen

Klicke auf die grüne Flagge.

Beobachte

- Startet das Programm?
 - Erscheint die Begrüßung?
 - Ist der Text richtig?
-

KAI erklärt

Auch dieses Scratch-Programm ist ein Algorithmus.

Jeder Block beschreibt einen Handlungsschritt.

Ausprobieren

Ändere das Programm.

Zum Beispiel:

- Begrüße die Besucher mit einem anderen Text.
 - Lasse KAI seinen Namen nennen.
 - Füge eine zweite Sprechblase hinzu.
-

Merke

Scratch verwendet Programmblöcke.

Die Blöcke beschreiben gemeinsam einen Algorithmus.

Algorithmen mit Python umsetzen

Dasselbe Problem

Im vorherigen Abschnitt hast du KAI mit Scratch programmiert.

Nun soll KAI **dieselbe Aufgabe** mit einer anderen Programmiersprache lösen.

KAI soll alle Besucher des Schulfestes begrüßen.

Der Algorithmus bleibt unverändert.

1. Warte, bis das Programm gestartet wird.
2. Gib eine Begrüßung aus.

Jetzt schreiben wir den Algorithmus als Python-Programm auf.

Python kennenlernen

Python ist eine textbasierte Programmiersprache.

Statt Programmblöcken werden Anweisungen als Text geschrieben.

Auch in Python gilt:

Jede Anweisung beschreibt einen Handlungsschritt.

Das erste Python-Programm

```
print("Willkommen auf unserem Schulfest!")
```

Was passiert?

Die Funktion `print()` gibt einen Text auf dem Bildschirm aus.

Alles, was zwischen den Anführungszeichen steht, erscheint später in der Ausgabe.

Programm testen

Starte das Programm.

Vergleiche die Ausgabe mit deinem Scratch-Programm.

Überlege

- Was ist gleich?
 - Was ist unterschiedlich?
-

Scratch und Python im Vergleich

Scratch	Python
Programmblöcke Blöcke werden zusammengesetzt. Beide beschreiben einen Algorithmus.	Text Anweisungen werden geschrieben. Beide beschreiben einen Algorithmus.

KAI erklärt

Für mich spielt es keine Rolle, ob ein Algorithmus aus Blöcken oder aus Text besteht.
Wichtig ist nur, dass die Anweisungen eindeutig sind.

Ausprobieren

Ändere den Begrüßungstext.

Zum Beispiel:

- „Herzlich willkommen!“
- „Schön, dass du da bist.“
- „Viel Spaß auf unserem Schulfest!“

Welche Ausgabe erscheint?

Merke

Python ist eine Programmiersprache.

Mit Python können Algorithmen so aufgeschrieben werden, dass ein Computer sie ausführen kann.

Übungen - Lineare Algorithmen

Ziel

In diesen Übungen festigst du die Grundlagen zu linearen Algorithmen.

Du sollst nicht nur richtige Abläufe erkennen, sondern auch erklären, **warum** ein Algorithmus funktioniert oder an welcher Stelle er verbessert werden muss.

1. Erkennen

Aufgabe 1 - Algorithmus oder nicht?

Entscheide jeweils, ob die Beschreibung als Algorithmus geeignet ist.

Begründe deine Entscheidung mit den Eigenschaften eines Algorithmus.

A

1. Öffne dein Heft.
2. Schreibe das Datum oben rechts auf die Seite.
3. Schreibe die Überschrift „Algorithmen“ darunter.

Bewertung: _____

Begründung: _____

B

1. Sei aufmerksam.
2. Arbeite ordentlich.
3. Gib dein Bestes.

Bewertung: _____

Begründung: _____

C

1. Nimm einen leeren Becher.
2. Stelle den Becher auf den Tisch.
3. Fülle 250 ml Wasser in den Becher.

Bewertung: _____

Begründung: _____

2. Fehler finden

Aufgabe 2 - KAI bereitet Kakao zu

KAI erhält folgende Anleitung:

1. Rühre 20 Sekunden um.
2. Stelle einen Becher auf den Tisch.
3. Gib zwei Teelöffel Kakaopulver hinein.
4. Fülle Milch ein.

KAI fragt:

„Was soll ich in Schritt 1 umrühren?“

Untersuche die Anleitung.

Finde mindestens **drei Probleme**.

1. _____
2. _____
3. _____

Überarbeite anschließend den Algorithmus.

3. Reihenfolge

Aufgabe 3 - Computerarbeitsplatz

Die folgenden Handlungsschritte sind durcheinandergeraten.

- Melde dich mit deinem Benutzerkonto an.
- Schalte den Monitor ein.
- Starte den Computer.
- Öffne die benötigte Anwendung.
- Bearbeite die Aufgabe.

Bringe die Schritte in eine sinnvolle Reihenfolge.

1. _____
 2. _____
 3. _____
 4. _____
 5. _____
- _____

4. Vollständigkeit

Aufgabe 4 - Brief verschicken

Ein Algorithmus lautet:

1. Schreibe den Brief.
2. Lege den Brief in einen Umschlag.
3. Wirf den Brief in einen Briefkasten.

Welche notwendigen Informationen oder Schritte fehlen?

Formuliere anschließend eine verbesserte Version.

5. Eindeutigkeit

Aufgabe 5 - Ungenaue Anweisungen

Verbessere die folgenden Anweisungen so, dass KAI möglichst wenig nachfragen muss.

A

„Nimm das Blatt.“

B

„Gehe zur Tür.“

C

„Gib etwas Wasser dazu.“

D

„Warte kurz.“

6. Einen Algorithmus entwickeln

Aufgabe 6 - Trinkflasche füllen

KAI soll eine leere Trinkflasche an einem Wasserhahn füllen.

Entwickle einen linearen Algorithmus.

Achte auf:

- Eindeutigkeit,
- Vollständigkeit,
- Ausführbarkeit,
- sinnvolle Reihenfolge.

START

1.

2.

3.

4.

5.

ENDE

7. Testen

Aufgabe 7 - Partnerprüfung

Tausche deinen Algorithmus aus Aufgabe 6 mit einer anderen Person.

Die andere Person darf **nur das ausführen oder gedanklich nachvollziehen, was tatsächlich dort steht.**

Markiert:

- fehlende Informationen,
- mehrdeutige Formulierungen,
- unnötige Schritte,
- falsche Reihenfolgen.

Überarbeite deinen Algorithmus danach.

Was hast du verändert?

Warum ist Version 2 besser?

8. Verschiedene Lösungen vergleichen

Aufgabe 8

Zwei Gruppen lösen dieselbe Aufgabe.

Algorithmus A

1. Nimm eine Flasche.
2. Öffne den Wasserhahn.
3. Halte die Flasche darunter.
4. Fülle die Flasche.
5. Schließe den Wasserhahn.

Algorithmus B

1. Stelle eine leere Trinkflasche unter den geschlossenen Wasserhahn.
2. Öffne die Flasche.
3. Öffne den Wasserhahn.
4. Fülle die Flasche bis etwa 2 cm unter den Rand.
5. Schließe den Wasserhahn.
6. Nimm die Flasche weg.

7. Verschließe die Flasche.

Beantworte:

1. Sind beide Algorithmen grundsätzlich verständlich?
2. Welcher Algorithmus ist eindeutiger?
3. Welcher lässt weniger Interpretationsspielraum?
4. Gibt es auch an Algorithmus B noch etwas zu verbessern?

Begründe deine Antworten.

9. Scratch und Python

Aufgabe 9 - Gleiche Idee, andere Darstellung

Ein Programm soll nacheinander drei Texte ausgeben:

1. „Willkommen!“
2. „Heute ist Schulfest.“
3. „Viel Spaß!“

Beschreibe zuerst den Algorithmus in Alltagssprache.

START

1.

2.

3.

ENDE

Setze ihn anschließend wahlweise

- in Scratch oder
- in Python

um.

Denkfrage

Was hat sich bei der Umsetzung verändert?

Was ist gleich geblieben?

Merke **Merke:** Die Programmiersprache verändert die Schreibweise der Umsetzung. Das zugrunde liegende Problem und der entwickelte Lösungsweg bleiben erhalten.

10. Fehler erklären

Aufgabe 10 - KAI macht genau das Richtige

Eine Schülerin sagt:

„KAI hat einen Fehler gemacht.“

KAI hat aber exakt die angegebenen Schritte ausgeführt und trotzdem ist das Ergebnis falsch.
Erkläre, wo der Fehler dann wahrscheinlich liegt.

11. Eigener Fehleralgorithmus

Entwickle absichtlich einen Algorithmus mit mindestens drei Fehlern.

Verwende unterschiedliche Fehlerarten.

Eine andere Gruppe soll die Fehler finden und erklären.

Mein fehlerhafter Algorithmus

Eingebaute Fehler

1.

 2.

 3.

-

12. Transfer

Aufgabe 12 - Vom Ablauf zum Problem

Wähle eine einfache Aufgabe aus dem Schulalltag.

Zum Beispiel:

- Material ausgeben,
- einen Computerarbeitsplatz vorbereiten,
- Bücher einsortieren,
- einen Versuchsplatz aufbauen.

Gehe systematisch vor.

Problem

Was soll erreicht werden?

Eingaben und Voraussetzungen

Was muss bekannt oder vorhanden sein?

Algorithmus

Welche Schritte führen zum Ziel?

Test

Wie kannst du prüfen, ob der Algorithmus funktioniert?

Verbesserung

Welche Änderung war nach dem Test notwendig?

Plus - Effizienz

Zwei Algorithmen führen zum gleichen richtigen Ergebnis.

Algorithmus A benötigt 15 Handlungsschritte.

Algorithmus B benötigt 8 Handlungsschritte.

Ist Algorithmus B automatisch besser?

Begründe.

Transfer

Bisher hast du Algorithmen für Aufgaben rund um das Schulfest entwickelt.
Nun überträgst du dein Wissen auf neue Situationen.

Aufgabe 1 - Frühstück vorbereiten

Beschreibe einen Algorithmus, mit dem ein Frühstück vorbereitet wird.

Achte darauf, dass dein Algorithmus

- eindeutig,
- vollständig,
- ausführbar und
- in der richtigen Reihenfolge

formuliert ist.

Aufgabe 2 - Papierflieger falten

Beschreibe möglichst genau, wie ein Papierflieger gefaltet wird.

Tausche anschließend deinen Algorithmus mit einer Mitschülerin oder einem Mitschüler.

Kann der Papierflieger allein anhand deiner Beschreibung gefaltet werden?

Verbessere deinen Algorithmus.

Aufgabe 3 - Der Schulweg

Beschreibe deinen Weg vom Klassenzimmer bis zum Haupteingang der Schule.

Welche Informationen müssen bekannt sein?

Wo könnten Missverständnisse entstehen?

KAI-Challenge

KAI soll einen Stuhl unter einen Tisch schieben.

Schreibe einen Algorithmus.

Ein anderes Team führt ihn genau aus.

Notiert alle Stellen, an denen Fragen entstehen.

Verbessert anschließend euren Algorithmus.

Aufgabe 4 - Ist das ein Algorithmus?

Beurteile die folgenden Aussagen.

Begründe deine Entscheidung.

- „Sei vorsichtig.“
- „Arbeite ordentlich.“
- „Öffne die Tür.“
- „Lege das Heft auf den Tisch.“
- „Räume dein Zimmer auf.“

Welche Aussagen eignen sich als Handlungsschritte eines Algorithmus?

Aufgabe 5 - Eigene Idee

Suche eine Alltagssituation, die sich als Algorithmus beschreiben lässt.

Erstelle einen möglichst eindeutigen Algorithmus.

Lass ihn von einer anderen Person überprüfen.

Projekt - KAI übernimmt eine neue Aufgabe

Der Auftrag

Beim Schulfest soll KAI weitere Aufgaben übernehmen.

Damit KAI erfolgreich arbeiten kann, benötigt er einen eindeutigen Algorithmus.

Ihr arbeitet als Entwicklungsteam.

Aufgabe

Wählt **eine** der folgenden Aufgaben aus.

Möglichkeit A - Getränke ausgeben

KAI soll einem Besucher einen Becher Wasser geben.

Möglichkeit B - Material ausgeben

KAI soll einem Gast einen Kugelschreiber ausgeben.

Möglichkeit C - Fundbüro

KAI soll einen gefundenen Gegenstand entgegennehmen und sicher ablegen.

Planung

Bevor ihr beginnt, besprecht gemeinsam:

- Was ist das Ziel?
 - Welche Schritte sind notwendig?
 - Fehlen Informationen?
 - Können alle Schritte ausgeführt werden?
-

Entwickelt euren Algorithmus

Schreibt euren Algorithmus auf.

Achtet besonders darauf, dass er

- eindeutig,
- vollständig,
- ausführbar und
- richtig geordnet

ist.

Test

Tauscht euren Algorithmus mit einem anderen Team.

Die andere Gruppe übernimmt die Rolle von KAI.

Sie führt jeden Schritt **genau so aus, wie er beschrieben wurde.**

Notiert:

- Wo entstehen Fragen?
 - Welche Schritte fehlen?
 - Welche Formulierungen sind unklar?
-

Überarbeitet euren Algorithmus

Verbessert euren Algorithmus mithilfe der Rückmeldungen.

Präsentation

Stellt euren fertigen Algorithmus kurz der Klasse vor.

Erklärt,

- welche Schwierigkeiten aufgetreten sind,
 - welche Änderungen ihr vorgenommen habt und
 - warum euer Algorithmus jetzt besser funktioniert.
-

KAI erklärt

Gute Algorithmen entstehen selten beim ersten Versuch.

Testen und Verbessern gehören zur Entwicklung dazu.

Reflexion

Kompetenzcheck

Das kann ich

Überprüfe dein Wissen und deine Fähigkeiten.

Aufgabe 1 - Algorithmus erkennen

Welche der folgenden Beschreibungen sind Algorithmen?

Begründe deine Entscheidung.

A

1. Öffne das Heft.
2. Schreibe die Überschrift.
3. Löse Aufgabe 1.

☐ Algorithmus

☐ Kein Algorithmus

B

1. Sei aufmerksam.
2. Arbeite ordentlich.
3. Gib dein Bestes.

☐ Algorithmus

☐ Kein Algorithmus

Aufgabe 2 - Fehler finden

Der folgende Algorithmus enthält Fehler.

1. Rühre um.
2. Nimm eine Tasse.
3. Gib Kakaopulver hinein.
4. Gieße Milch ein.

Beschreibe mindestens zwei Probleme.

Aufgabe 3 - Algorithmus entwickeln

Schreibe einen Algorithmus.

Aufgabe:

KAI soll einen Brief in einen Briefkasten einwerfen.

Achte auf

- Eindeutigkeit,
- Vollständigkeit,
- richtige Reihenfolge und
- Ausführbarkeit.

Aufgabe 4 - Darstellungen vergleichen

Scratch und Python können denselben Algorithmus darstellen.

Erkläre mit eigenen Worten.

Selbsteinschätzung

Ich kann ...

Aussage	:-)	☹	☹
einen Algorithmus erkennen	☐	☐	☐
einen Algorithmus entwickeln	☐	☐	☐
Fehler in Algorithmen finden	☐	☐	☐
Algorithmen verbessern	☐	☐	☐
Algorithmen in Scratch darstellen	☐	☐	☐
Algorithmen in Python darstellen	☐	☐	☐

Geschafft?

Wenn du die meisten Aufgaben sicher lösen konntest, bist du bereit für das nächste Kapitel.

Dort lernst du Algorithmen kennen, die Entscheidungen treffen.

Projektstart - Von der Übung zur Komplexaufgabe

Jetzt beginnt das eigentliche Projekt

Bisher hast du grundlegende Eigenschaften von Algorithmen wiederholt und verschiedene Darstellungen kennengelernt.

Jetzt arbeitest du mit einer Partnerin, einem Partner oder in einem kleinen Team an einer **eigenen informatischen Lösung**.

Merke **Wichtig**: Nicht die Programmiersprache steht im Mittelpunkt. Entscheidend ist, wie ihr ein Problem versteht, eine Lösung plant, umsetzt, testet und verbessert.

Was ist ein Projekt?

Ein Projekt hat

- ein klares Ziel,
- einen Anfang und ein Ende,
- mehrere Arbeitsschritte,
- Aufgaben, die geplant werden müssen,
- ein Ergebnis, das vorgestellt werden kann.

In diesem Projekt arbeitet ihr möglichst selbstständig.

Der Projektauftrag

Entwickelt eine digitale Lösung für eine **einfache Problemstellung**.

Mögliche Bereiche sind zum Beispiel:

- ein kleines Computerspiel,
- eine Simulation,
- eine grafische Anwendung,
- ein digitaler Helfer für das Schulfest,
- eine kleine Steuerung oder Robotik-Aufgabe, wenn passende Technik vorhanden ist.

Eure Lehrkraft kann einen gemeinsamen Projektrahmen vorgeben.

Unser Team

Teamname: _____

Mitglieder: _____

Unsere erste Idee

Welches Problem möchten wir lösen?

Was soll unsere Lösung am Ende können?

Rollen im Team

Rollen helfen bei der Organisation. Sie sind **keine festen Berufe**. Ihr dürft sie während des Projekts wechseln.

Mögliche Rollen:

- Planung
- Umsetzung
- Test
- Dokumentation
- Präsentation

Unsere erste Rollenverteilung

Aufgabe	Verantwortlich
Planung	
Umsetzung	
Test	
Dokumentation	
Präsentation	

Der Qualitätskreislauf

Eine gute Lösung entsteht selten sofort.

Planen



Umsetzen



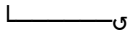
Testen



Bewerten



Verbessern



Diesen Kreislauf werdet ihr während des Projekts mehrfach durchlaufen.

Projektregeln

Wir verpflichten uns,

- Entscheidungen gemeinsam zu treffen,
- Arbeitsergebnisse regelmäßig zu speichern,

- Änderungen nachvollziehbar zu dokumentieren,
 - Probleme früh anzusprechen,
 - Lösungen zu testen,
 - andere Teammitglieder zu unterstützen.
-

Startcheck

- ☐ Wir kennen unser Problem.
- ☐ Wir haben ein gemeinsames Ziel.
- ☐ Wir wissen, wer im Team arbeitet.
- ☐ Wir kennen den Zeitrahmen.
- ☐ Wir wissen, wo unsere Dateien gespeichert werden.
- ☐ Wir haben einen ersten Plan.

Wenn ihr alle Punkte beantworten könnt, beginnt die Problemanalyse.

Problemanalyse - Was müssen wir eigentlich lösen?

Nicht sofort programmieren

Ein häufiger Fehler bei Projekten ist:

Aufgabe lesen → Programmierumgebung öffnen → loslegen

Das wirkt schnell, führt aber oft zu unnötiger Arbeit.

Bevor ihr etwas umsetzt, müsst ihr genau verstehen, **welches Problem ihr lösen wollt**.

1. Problem beschreiben

Formuliert das Problem in einem Satz.

Unser Problem ist:

2. Ziel beschreiben

Wie sieht ein erfolgreiches Ergebnis aus?

Am Ende soll unsere Lösung ...

3. Zielgruppe

Wer soll eure Lösung benutzen?

- ☐ Schülerinnen und Schüler
- ☐ Lehrkräfte
- ☐ Besucher eines Schulfestes
- ☐ Kinder
- ☐ andere: _____

Was muss eure Lösung für diese Zielgruppe berücksichtigen?

4. Eingaben, Verarbeitung, Ausgaben und Speicherung

Das EVAS-Prinzip kennst du bereits.

Übertrag es auf euer Projekt.

Eingabe

Welche Daten oder Aktionen erhält eure Lösung?

Verarbeitung

Was muss mit diesen Daten geschehen?

Ausgabe

Was soll als Ergebnis sichtbar oder hörbar werden?

Speicherung

Welche Informationen müssen während oder nach der Nutzung gespeichert werden?

5. Anforderungen

Eine **Anforderung** beschreibt, was eure Lösung leisten soll.

Muss-Anforderungen

Ohne diese Funktionen ist das Projektziel nicht erreicht.

1.

2.

3.

Kann-Anforderungen

Diese Funktionen wären sinnvoll, sind aber nicht zwingend notwendig.

1.

2.

3.

6. Grenzen festlegen

Nicht alles, was interessant wäre, muss in euer Projekt.

Was gehört **bewusst nicht** zu eurer ersten Version?

7. Prüfen

Tauscht eure Problemanalyse mit einem anderen Team.

Das andere Team beantwortet:

- Verstehen wir das Problem?
- Ist das Ziel eindeutig?
- Sind die Muss-Anforderungen überprüfbar?
- Fehlen wichtige Informationen?

Rückmeldung des anderen Teams

Das ändern wir danach

Merke **Merke:** Je besser ein Problem verstanden ist, desto einfacher lässt sich eine passende Lösung entwickeln.

Lösungsentwurf und Planung

Vom Problem zur Lösung

Ihr kennt jetzt das Problem und die Anforderungen.

Jetzt entwickelt ihr einen Lösungsweg.

Noch ist keine perfekte Lösung notwendig.

1. Lösungsideen sammeln

Notiert mindestens zwei mögliche Lösungsansätze.

Idee A

Idee B

2. Ideen vergleichen

Kriterium	Idee A	Idee B
erfüllt die Muss-Anforderungen		
verständlich		
im Zeitrahmen umsetzbar		
gut testbar		
gut erweiterbar		

Unsere Entscheidung

Wir wählen:

Begründung

3. Algorithmus entwerfen

Beschreibt die wichtigsten Schritte zunächst unabhängig von einer Programmiersprache.

START

1.

2.

3.

4.

5.

ENDE

Wenn eure Lösung Entscheidungen oder Wiederholungen benötigt, kennzeichnet diese ebenfalls.

4. Umsetzung wählen

Welche Werkzeuge verwendet ihr?

☐ Scratch

☐ Python

☐ andere Software: _____

☐ Hardware/Robotik: _____

Warum passt dieses Werkzeug zu eurer Lösung?

5. Projekt in kleine Aufgaben zerlegen

Große Probleme werden leichter, wenn sie in kleine Teilprobleme zerlegt werden.

Nr.	Teilaufgabe	Verantwortlich	Status
1			offen
2			offen
3			offen
4			offen
5			offen

6. Erste lauffähige Version

Plant zuerst eine **kleine Version, die bereits funktioniert.**

Unsere Version 1 kann:

Noch nicht enthalten ist:

Merke **Inkrementell arbeiten:** Erst eine kleine funktionierende Lösung erstellen.
Danach Schritt für Schritt erweitern.

Planungscheck

- ☐ Unsere Lösung erfüllt die Muss-Anforderungen.
- ☐ Wir haben einen Algorithmus oder Ablaufentwurf.
- ☐ Wir haben passende Werkzeuge gewählt.
- ☐ Das Projekt ist in Teilaufgaben zerlegt.
- ☐ Wir wissen, was Version 1 können soll.

Umsetzung - Aus dem Entwurf wird eine funktionierende Lösung

Kleine Schritte

Setzt nicht sofort alle geplanten Funktionen um.

Arbeitet in kleinen Schritten:

kleine Änderung

↓

ausführen

↓

prüfen

↓

speichern

↓

nächste Änderung

Version 1

Welche Funktion setzt ihr zuerst um?

Ergebnis

- ☐ funktioniert
- ☐ funktioniert teilweise
- ☐ funktioniert noch nicht

Beobachtung

Entwicklungsprotokoll

Dokumentiert wichtige Änderungen.

Version	Änderung	Ergebnis des Tests
0.1		
0.2		
0.3		
0.4		

Wenn etwas nicht funktioniert

Nicht wahllos ändern.

Geht systematisch vor:

1. Was sollte passieren?
 2. Was passiert tatsächlich?
 3. An welcher Stelle unterscheiden sich Erwartung und Ergebnis?
 4. Welche Anweisung könnte dafür verantwortlich sein?
 5. Ändert **eine** Sache.
 6. Testet erneut.
-

KAI fragt

„Welche Anweisung soll ich als Nächstes ausführen?“

Wenn ihr diese Frage nicht eindeutig beantworten könnt, ist euer Lösungsentwurf möglicherweise noch nicht genau genug.

Kommunikation im Team

Besprecht regelmäßig:

- Was ist fertig?
- Wo gibt es Probleme?
- Wer benötigt Unterstützung?
- Welche Aufgabe kommt als Nächstes?

Unser aktueller Stand

Unser größtes Problem

Unser nächster Schritt

Zwischenstand sichern

- ☐ Alle aktuellen Dateien gespeichert.
- ☐ Dateinamen sind verständlich.
- ☐ Teammitglieder kennen den Speicherort.
- ☐ Entwicklungsprotokoll aktualisiert.

Testen und Verbessern

„Es läuft“ reicht nicht

Ein Programm kann starten und trotzdem Fehler enthalten.

Testen bedeutet deshalb nicht nur:

„Startet das Programm?“

Sondern:

„Erfüllt die Lösung unsere Anforderungen?“

1. Testfälle entwickeln

Ein Testfall beschreibt:

- eine Ausgangssituation,
- eine Eingabe oder Aktion,
- das erwartete Ergebnis.

Beispiel

Eingabe/Aktion	Erwartetes Ergebnis	Tatsächliches Ergebnis
Start	Begrüßung erscheint	Begrüßung erscheint

Unsere Testfälle

	Nr.	Eingabe/Aktion	Erwartetes Ergebnis	Tatsächliches Ergebnis	bestanden
	1				
	2				
	3				
	4				
	5				

2. Muss-Anforderungen testen

Geht eure Muss-Anforderungen aus der Problemanalyse durch.

Anforderung	erfüllt	teilweise	nicht erfüllt
1			
2			
3			

3. Fremdtest

Lasst eure Lösung von einer Person testen, die nicht an der Umsetzung beteiligt war.
Die Testperson darf zunächst keine Erklärung erhalten.

Beobachtet

- Versteht sie die Bedienung?
- Funktionieren alle wichtigen Wege?
- Gibt es unerwartete Ergebnisse?
- Sind Hinweise verständlich?

Rückmeldung

4. Fehler priorisieren

Nicht jeder Fehler ist gleich wichtig.

Kritisch

Die Kernfunktion funktioniert nicht.

Wichtig

Die Lösung funktioniert, aber eine wichtige Funktion ist fehlerhaft.

Klein

Die Funktion ist korrekt, aber Darstellung oder Bedienung können verbessert werden.

Fehler	Priorität	Änderung
--------	-----------	----------

5. Qualitätskreislauf

Planen

↓

Umsetzen

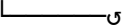
↓

Testen

↓

Bewerten

↓

Verbessern


Welche Verbesserung hat euer Projekt am stärksten verändert?

Warum?

Dokumentation - Den Lösungsweg nachvollziehbar machen

Dokumentation ist Teil des Projekts

Eine Dokumentation beschreibt nicht nur das fertige Ergebnis.

Sie zeigt auch, **wie** die Lösung entstanden ist.

Eure Projektdokumentation

Sie sollte mindestens enthalten:

1. Projektziel

Welches Problem habt ihr bearbeitet?

2. Anforderungen

Welche Muss-Anforderungen sollte eure Lösung erfüllen?

3. Lösungsentwurf

Wie habt ihr das Problem gelöst?

4. Umsetzung

Welche Werkzeuge habt ihr verwendet?

5. Test

Wie habt ihr überprüft, ob eure Lösung funktioniert?

6. Verbesserungen

Welche wichtigen Änderungen entstanden während des Projekts?

7. Ergebnis

Was kann eure fertige Lösung?

Entscheidungen dokumentieren

Notiert mindestens eine wichtige Entscheidung.

Entscheidung

Welche Alternativen gab es?

Warum habt ihr euch so entschieden?

Schwierigkeiten

Welche Schwierigkeit war besonders wichtig?

Wie habt ihr sie gelöst?

Zusammenarbeit

Wie habt ihr die Aufgaben verteilt?

Was hat im Team gut funktioniert?

Was würdet ihr beim nächsten Projekt ändern?

Dokumentationscheck

- ☐ Problem beschrieben
- ☐ Anforderungen genannt
- ☐ Lösungsweg nachvollziehbar
- ☐ verwendete Werkzeuge genannt
- ☐ Tests beschrieben
- ☐ wichtige Verbesserungen dokumentiert
- ☐ Ergebnis erklärt
- ☐ Quellen angegeben, falls externe Inhalte verwendet wurden

Präsentation und Reflexion

Euer Ergebnis vorstellen

Eine gute Präsentation zeigt nicht nur das fertige Produkt.

Sie erklärt auch den Weg dorthin.

Präsentationsstruktur

1. Problem

Was sollte gelöst werden?

2. Ziel

Was sollte eure Lösung können?

3. Lösungsweg

Wie seid ihr vorgegangen?

4. Ergebnis

Zeigt eure funktionierende Lösung.

5. Test und Verbesserung

Welche Fehler oder Schwächen habt ihr gefunden?

6. Erkenntnis

Was habt ihr aus dem Projekt gelernt?

Präsentationsregeln

- kurze und verständliche Erklärungen,
 - nicht den gesamten Quellcode vorlesen,
 - wichtige Teile praktisch zeigen,
 - alle Teammitglieder beteiligen,
 - Fragen beantworten können.
-

Vorbereitung

Unser wichtigster Satz zum Problem

Das müssen wir unbedingt zeigen

Unser wichtigster Test

Unsere größte Verbesserung

Reflexion

Über das Ergebnis

Wie gut erfüllt eure Lösung die Muss-Anforderungen?

Was funktioniert besonders gut?

Was würdet ihr technisch noch verbessern?

Über den Lösungsweg

Welche Entscheidung war besonders wichtig?

Wann musstet ihr euren ursprünglichen Plan ändern?

Was hat euch beim Finden einer Lösung geholfen?

Über die Zusammenarbeit

Was hat im Team gut funktioniert?

Wo gab es Schwierigkeiten?

Was würdet ihr beim nächsten Projekt anders organisieren?

Über das Lernen

Vervollständige:

Vor dem Projekt dachte ich, dass ...

Jetzt weiß ich, dass ...

Beim nächsten Problem werde ich zuerst ...

Abschluss

Merke **Gute Problemlösung bedeutet nicht, sofort die richtige Antwort zu kennen. Gute Problemlösung bedeutet, systematisch einen Weg zur Lösung zu entwickeln, ihn zu prüfen und zu verbessern.**

Arbeitsheft - Komplexaufgabe 1: Lineare Algorithmen

Ausgangssituation

Für das Schulfest soll ein Programm entwickelt werden, das Eingaben verarbeitet und Ergebnisse berechnet.

Kompetenzziele

Lineare Algorithmen lesen, entwickeln und in Pseudocode, PAP und Python darstellen.

Aufgabe 1

Beschreibe den Ablauf zur Berechnung des Gesamtpreises aus Anzahl und Einzelpreis.

Aufgabe 2

Erstelle ein PAP für die Berechnung.

Aufgabe 3

Formuliere den Algorithmus als Pseudocode.

Aufgabe 4

Übertrage den Algorithmus nach Python.

Transfer

Entwickle einen Algorithmus zur Berechnung des Durchschnitts aus drei Messwerten.

Kompetenzcheck

Erkläre den Unterschied zwischen Variable, Zuweisung und Ausgabe.